1D + 2D RANGE SUM QUERY

Complete DSA Guide • Brute Force • Prefix Sum • Fenwick Tree • Segment Tree

Yes — Range Sum Query is not limited to Brute Force. The right algorithm depends on whether the data changes and how many queries you need. This guide covers both 1D arrays and 2D matrices.

Quick rule: for many queries on data that does not change, Prefix Sum is usually simplest and fastest. If values are updated, Fenwick Tree or Segment Tree is more suitable.

PART A — 1D RANGE SUM QUERY

1. Problem and example

Array: [2, 4, 6, 8, 10]. Query L=1, R=3 asks for indices 1 through 3, inclusive.

4 + 6 + 8 = 18

2. Approach 1: Brute Force

Visit every index from L to R and add the value. This is easy to understand and useful for a small number of queries.

Java code — 1D Brute Force

```
int[] arr = {2, 4, 6, 8, 10};
int L = 1, R = 3;
int sum = 0;

for (int i = L; i <= R; i++) {
    sum += arr[i];
}
System.out.println(sum); // 18
```

Complexity: O(R–L+1) per query; worst case O(n). Extra space O(1).

3. Approach 2: Prefix Sum

Create prefix[i] = sum of arr[0] through arr[i]. For a query [L,R], subtract the sum before L from the sum through R.

Formula: if L=0, answer=prefix[R]; otherwise answer=prefix[R]–prefix[L–1].

Complete Java code — 1D Prefix Sum

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 4, 6, 8, 10};
        int n = arr.length;

        int[] prefix = new int[n];
        prefix[0] = arr[0];

        for (int i = 1; i < n; i++) {
            prefix[i] = prefix[i - 1] + arr[i];
        }

        int L = 1, R = 3;
        int sum = (L == 0)
                ? prefix[R]
                : prefix[R] - prefix[L - 1];

        System.out.println(sum); // 18
    }
}
```

Complexity: Build O(n), each query O(1), extra space O(n). Great for many queries when the array is static.

4. Approach 3: Fenwick Tree (Binary Indexed Tree)

Use this when values can be updated and you need prefix/range sums. A Fenwick Tree supports point update and prefix sum in O(log n); range sum is prefix(R)–prefix(L–1).

Java code — Fenwick Tree (1-based internal indexing)

```
class Fenwick {
    int n;
    long[] bit;
```

```
    Fenwick(int n) {
        this.n = n;
        bit = new long[n + 1];
    }

    // Add delta to array position index (0-based).
    void add(int index, long delta) {
        for (int i = index + 1; i <= n; i += i & -i) {
            bit[i] += delta;
        }
    }

    // Sum of array positions 0 through index.
    long prefixSum(int index) {
        long sum = 0;
        for (int i = index + 1; i > 0; i -= i & -i) {
            sum += bit[i];
        }
        return sum;
    }

    // Inclusive range sum [left, right].
    long rangeSum(int left, int right) {
        long rightSum = prefixSum(right);
        long leftSum = (left == 0) ? 0 : prefixSum(left - 1);
        return rightSum - leftSum;
    }
}
```

How to use: create Fenwick(n), then call add(i, arr[i]) for every initial element. To change arr[i] from oldValue to newValue, call add(i, newValue–oldValue).

Complexity: Build by n updates O(n log n) with this simple initialization; point update O(log n); range query O(log n); space O(n).

5. Approach 4: Segment Tree

A Segment Tree stores sums for intervals in a tree. It supports point updates and range sum queries in O(log n), with O(n) storage (commonly implemented using an array of about 4n slots). It is more flexible than a Fenwick Tree for many other query types, such as minimum or maximum.

When to choose: dynamic array + range queries, especially when the problem may later need operations other than sum. For a static array, Prefix Sum is usually simpler.

PART B — 2D RANGE SUM QUERY

6. Problem and example

Matrix (zero-based row and column indices):

	c=0	c=1	c=2	c=3
r=0	1	2	3	4
r=1	5	6	7	8
r=2	9	1	2	3
r=3	4	5	6	7

Query rectangle: top-left (r1=1,c1=1), bottom-right (r2=2,c2=2), inclusive. Selected values are 6, 7, 1, 2; answer = 16.

7. Approach 1: Brute Force

Use nested loops: the outer loop visits each requested row; the inner loop visits each requested column in that row.

Java code — 2D Brute Force

```
int[][] a = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 1, 2, 3},
    {4, 5, 6, 7}
};
int r1 = 1, c1 = 1, r2 = 2, c2 = 2;
int sum = 0;
```

```
for (int i = r1; i <= r2; i++) {
    for (int j = c1; j <= c2; j++) {
        sum += a[i][j];
    }
}
System.out.println(sum); // 16
```

Complexity: O(height × width) per query; worst case O(rows × columns). Extra space O(1).

8. Approach 2: 2D Prefix Sum

Define P[i][j] as the sum of the original matrix rectangle from (0,0) through (i–1,j–1). Use a padded prefix matrix with one extra row and one extra column, both initialized to zero.

Build formula (1-based prefix coordinates):

P[i][j] = a[i–1][j–1] + P[i–1][j] + P[i][j–1] – P[i–1][j–1]

The top-left overlap is counted in both the top and left prefix regions, so subtract it once.

Rectangle query formula:

sum = P[r2+1][c2+1] – P[r1][c2+1] – P[r2+1][c1] + P[r1][c1]

Why add the last term? The top-left overlap was subtracted twice, so add it back once.

Complete Java code — 2D Prefix Sum

```
public class Main {
    public static void main(String[] args) {
        int[][] a = {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 1, 2, 3},
            {4, 5, 6, 7}
        };

        int rows = a.length, cols = a[0].length;
        int[][] p = new int[rows + 1][cols + 1];

        // Build the padded 2D prefix sum.
        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= cols; j++) {
                p[i][j] = a[i-1][j-1]
                        + p[i-1][j]
                        + p[i][j-1]
                        - p[i-1][j-1];
            }
        }

        int r1 = 1, c1 = 1, r2 = 2, c2 = 2;

        int sum = p[r2+1][c2+1]
                - p[r1][c2+1]
                - p[r2+1][c1]
                + p[r1][c1];

        System.out.println(sum); // 16
    }
}
```

Complexity: Build O(rows × columns), each query O(1), extra space O(rows × columns). Best for many queries on a matrix that does not change.

9. Approach 3: 2D Fenwick Tree

A 2D Fenwick Tree (2D BIT) is useful when matrix cells are updated and rectangle-sum queries are required. It supports point update and prefix-rectangle sum in O(log rows × log columns). A rectangle sum is obtained using four prefix queries and inclusion-exclusion.

Core idea: update one cell by a delta; query the sum from (0,0) to (r,c); combine four such prefix sums to get any rectangle.

Java code — 2D Fenwick Tree

```
class Fenwick2D {
    int rows, cols;
    long[][] bit;

    Fenwick2D(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        bit = new long[rows + 1][cols + 1];
    }
```

```
    // Add delta to cell (row, col), both 0-based.
    void add(int row, int col, long delta) {
        for (int i = row + 1; i <= rows; i += i & -i) {
            for (int j = col + 1; j <= cols; j += j & -j) {
                bit[i][j] += delta;
            }
        }
    }

    // Sum rectangle (0,0) through (row,col).
    long prefix(int row, int col) {
        if (row < 0 || col < 0) return 0;
        long sum = 0;
        for (int i = row + 1; i > 0; i -= i & -i) {
            for (int j = col + 1; j > 0; j -= j & -j) {
                sum += bit[i][j];
            }
        }
        return sum;
    }

    // Inclusive rectangle sum.
    long rangeSum(int r1, int c1, int r2, int c2) {
        return prefix(r2, c2)
             - prefix(r1 - 1, c2)
             - prefix(r2, c1 - 1)
             + prefix(r1 - 1, c1 - 1);
    }
}
```

Initialization: For every original matrix cell a[i][j], call add(i,j,a[i][j]). For a later assignment of a cell from oldValue to newValue, call add(i,j,newValue–oldValue).

Complexity: Point update O(log rows × log columns); rectangle query O(log rows × log columns); space O(rows × columns).

10. Approach 4: 2D Segment Tree

A 2D Segment Tree is another dynamic structure for point updates and rectangle queries. It is more complex and uses more memory than a 2D Prefix Sum or 2D Fenwick Tree. It is useful for advanced problems or when the required operation needs Segment Tree flexibility.

For ordinary sum queries, prefer 2D Prefix Sum if the matrix is static. If cells change, a 2D Fenwick Tree is often a simpler dynamic option.

PART C — WHICH ALGORITHM SHOULD YOU USE?

Situation	Recommended approach	Why
1D, one/few queries; no updates	Brute Force	Simple; no preprocessing
1D, many queries; no updates	Prefix Sum	O(1) per query
1D, point updates + sum queries	Fenwick / Segment Tree	Updates and queries O(log n)
2D, one/few queries; no updates	Brute Force	Direct rectangle traversal
2D, many queries; no updates	2D Prefix Sum	O(1) per rectangle query
2D, point updates + rectangle sums	2D Fenwick Tree	Dynamic updates and queries
Advanced 2D operations	2D Segment Tree	More flexible, more complex

PART D — REVISION AND PRACTICE

Remember these differences:

• 1D query is an interval [L,R]. 2D query is a rectangle [r1..r2] × [c1..c2].

• Prefix Sum is excellent for static data: build once, query many times.

• Fenwick Tree and Segment Tree are designed for changing data as well as queries.

• Brute Force is a valid algorithm, not the only algorithm.

Practice questions

Q1. For [3,5,7,9,11], find the 1D sum from L=1 to R=3. **Answer:** 21.

Q2. Why does a 1D prefix query subtract prefix[L−1]? **Answer:** To remove the sum before the requested range.

Q3. For the 2D example, query rows 1..2 and columns 1..2. **Answer:** 16.

Q4. Which 2D approach supports point updates efficiently? **Answer:** 2D Fenwick Tree or 2D Segment Tree.

Q5. Why is Prefix Sum not automatically the best choice when values change? **Answer:** Updating one value can require changing many later prefix values.

Final takeaway: learn Brute Force first to understand the range, then Prefix Sum for fast static queries, and then Fenwick/Segment Tree for dynamic queries.